

Mastering Terraform Remote State For Absolute Beginners

Terraform state is the memory of your cloud infrastructure, and keeping it only on your laptop is a ticking time bomb. Learning remote state early saves you from accidentally destroying production servers and helps your team collaborate smoothly.

What you'll learn:

- 1 Understanding the Terraform State File
- 2 Setting Up an S3 Bucket for Remote State
- 3 Configuring the S3 Backend in Code
- 4 Preventing Conflicts with State Locking

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Understanding the Terraform State File

Terraform creates a file named `terraform.tfstate` every time you build infrastructure. Think of it as a receipt that matches your code to actual cloud servers. If this file gets deleted or corrupted, Terraform loses track of everything you built, leaving you with orphaned cloud resources.

```
$cat terraform.tfstate
```

Cloud & DevOps Daily

Terraform Remote State: Why You Should Never Store It Locally

Thursday, September 10, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Setting Up an S3 Bucket for Remote State

Instead of keeping the receipt on your laptop, you store it in a secure cloud bucket like AWS S3. This ensures the file is safely backed up and accessible from anywhere. Every team member now reads from the exact same single source of truth.

```
$aws s3 mb s3://my-terraform-state-bucket-xyz
```

Cloud & DevOps Daily

Terraform Remote State: Why You Should Never Store It Locally

Thursday, September 10, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

Configuring the S3 Backend in Code

You need to tell your Terraform configuration block where to find your remote storage bucket. By adding a backend block inside your main.php or main.tf file, Terraform automatically redirects state saving from your local disk to the cloud.

```
terraform {  
  backend "s3" {  
    bucket = "my-terraform-state-bucket-xyz"  
    key     = "prod/terraform.tfstate"  
    region = "us-east-1"  
  }  
}
```

Cloud & DevOps Daily

Terraform Remote State: Why You Should Never Store It Locally

Thursday, September 10, 2026

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

Preventing Conflicts with State Locking

When multiple people use remote state, two people might try to apply changes at the same time. DynamoDB acts as a digital traffic light that locks the state file while one person is running updates, preventing broken cloud infrastructure.

```
$aws dynamodb create-table --table-name terraform-locks --attribute-definitions AttributeName=LockID,
```


Cloud & DevOps Daily

Terraform Remote State: Why You Should Never Store It Locally

Thursday, September 10, 2026

See Key Takeaways on next page >> linkedin.com/in/mrmanrajsingh

Key Takeaways

- + Always enable versioning on your S3 state bucket
- + Use DynamoDB for state locking from day one
- + Never share your AWS credentials in plain text
- + Run terraform plan before applying any remote changes
- + Back up your state file manually before major migrations

Watch Out For

- ! Forgetting to add terraform.tfstate to your .gitignore file
- ! Allowing two teammates to run terraform apply without state locking
- ! Deleting the S3 bucket while active infrastructure still exists

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mrmanrajsingh